Selio

DOCUMENT DE MISE EN SERVICE · NOVEMBRE 2026

Le PC de Selio sous Windows

Pas à pas · novembre 2026 · pour la machine de Marseille (Windows 10/11 64 bits, 32 Go de mémoire vive, carte NVIDIA de 16 Go)

Selio · PC-WINDOWS-IA.md

On garde Windows. Tout tient sur ce PC :

- **l'IA** : Ollama, installé directement sous Windows (accès le plus direct à la carte graphique), avec ses trois modèles ;
- **le serveur Selio** (base de données, API, worker, photos) : dans Docker Desktop, qui fait tourner Linux à l'intérieur de Windows, sans réinstaller le PC ;
- **le tunnel Cloudflare** : rend l'API et les photos joignables en HTTPS depuis Internet **sans ouvrir aucun port sur la box**.

Le site lui-même reste sur Netlify. Durée totale : une demi-journée, dont une bonne partie en téléchargements.

PARTIE	QUOI	DURÉE
1	Préparer Windows (alimentation, mises à jour, pilote NVIDIA)	30 min
2	Installer l'IA : Ollama et les trois modèles	45 min
3	Faire tout redémarrer seul après une coupure	20 min
4	Installer Docker Desktop, récupérer Selio, remplir les clés	1 h
5	Nom de domaine et tunnel Cloudflare	30 min
6	Démarrer et vérifier	20 min
7	Mesurer l'IA (et m'envoyer les chiffres)	30 min
8	Au quotidien, et en cas de problème	—

Les clés ne voyagent jamais. Tu les crées et tu les ranges sur ce PC, dans `deploy\.env` (partie 4). Tu ne les colles ni dans une conversation, ni dans un e-mail, ni sur GitHub. Ce que tu peux m'envoyer : le rapport de `verifier-cles.mjs` (il ne contient aucun secret) et les chiffres de la partie 7.

Partie 1 · Préparer Windows

1.1 Mettre Windows à jour. Paramètres › Windows Update › Rechercher des mises à jour, installer, redémarrer, recommencer jusqu'à ce qu'il n'y ait plus rien.

1.2 Ne jamais mettre le PC en veille. Ouvrir **PowerShell en administrateur** (clic droit sur le menu Démarrer › Terminal (administrateur)) :

```
powercfg /change standby-timeout-ac 0     # jamais de veille sur secteur
powercfg /change hibernate-timeout-ac 0   # jamais de veille prolongée
powercfg /change monitor-timeout-ac 10    # l'écran peut s'éteindre, lui
```

1.3 Limiter les redémarrages surprise. Paramètres › Windows Update › Options avancées › **Heures d'activité** : manuel, de 7 h à 1 h (le maximum autorisé). Windows redémarrera pour ses mises à jour en dehors de ces heures ; la partie 3 fait tout repartir seul.

1.4 Redémarrer seul après une coupure de courant. Au démarrage du PC, entrer dans le BIOS (touche Suppr ou F2) et chercher l'option **« Restore on AC Power Loss »** (selon la carte mère : « AC Back », « Power On After Power Fail »). La mettre sur **Power On**. Enregistrer et quitter.

1.5 Pilote NVIDIA à jour. Installer la dernière version depuis l'application NVIDIA ou nvidia.com (pilote « Game Ready » ou « Studio », peu importe). Puis, dans PowerShell :

```
nvidia-smi --query-gpu=name,memory.total,driver_version --format=csv
```

On doit voir la carte et **16 Go** environ (16376 MiB). Note le nom exact de la carte : il ira dans le tableau des mesures (partie 7).

1.6 Chiffrer le disque (recommandé). Le PC va contenir la base des clients et leurs photos. Windows Pro : Paramètres › Confidentialité et sécurité › Chiffrement de l'appareil, ou Panneau de configuration › **BitLocker**. Garde la clé de récupération hors du PC (sur papier, avec la copie des clés de Selio).

Partie 2 · Installer l'IA : Ollama et les trois modèles

Les trois modèles, tous servis par Ollama sur ce PC (détail : `docs/OLLAMA-SETUP.md`) :

RÔLE	MODÈLE	TAILLE SUR LE DISQUE
Mon IA perso (texte : réponses aux acheteurs, annonces, aide)	celui que tu choisis, ex. `qwen2.5:7b-instruct`	≈ 5 Go
Modèle 1, **les photos** (Studio)	`qwen3-vl:8b`	≈ 6 Go
Modèle 2, **les prix** (comparaison des ventes)	`qwen3-embedding:0.6b`	≈ 0,6 Go

2.1 (Facultatif) Ranger les modèles sur un autre disque. Par défaut ils vont dans `C:\Users\<toi>\.ollama\models` . Si un disque `D:` a plus de place, crée le dossier `D:\selio\ollama-models` ; on le déclare à l'étape 2.3.

2.2 Installer Ollama. Télécharger `OllamaSetup.exe` sur **ollama.com/download**, l'installer (pas besoin d'être administrateur). Une icône de lama apparaît près de l'horloge : Ollama tourne.

2.3 Régler Ollama. Ollama lit ses réglages dans les variables d'environnement de Windows. Dans PowerShell (**sans** administrateur) :

```
[Environment]::SetEnvironmentVariable("OLLAMA_HOST", "127.0.0.1:11434", "User")   # n'écoute que sur ce PC
[Environment]::SetEnvironmentVariable("OLLAMA_MAX_LOADED_MODELS", "3", "User")    # garde les 3 modèles en mémoire
[Environment]::SetEnvironmentVariable("OLLAMA_KEEP_ALIVE", "30m", "User")         # 30 min avant de décharger un modèle inutilisé
[Environment]::SetEnvironmentVariable("OLLAMA_NUM_PARALLEL", "2", "User")         # 2 demandes à la fois par modèle
# Seulement si tu as fait l'étape 2.1 :
[Environment]::SetEnvironmentVariable("OLLAMA_MODELS", "D:\selio\ollama-models", "User")
```

Puis **quitter Ollama** (clic droit sur le lama › Quit Ollama) et le relancer depuis le menu Démarrer : il ne lit ses réglages qu'au démarrage.

2.4 Télécharger les trois modèles. Dans PowerShell (remplace le premier nom par le modèle de texte choisi) :

```
ollama pull qwen2.5:7b-instruct
ollama pull qwen3-vl:8b
ollama pull qwen3-embedding:0.6b
ollama list
```

`ollama list` doit afficher les trois. Le nom exact du premier ira dans `OLLAMA_MODEL` (partie 4).

2.5 Premier essai.

```
ollama run qwen2.5:7b-instruct "Rédige une annonce Vinted courte pour un pull à pois Tommy
Hilfiger, taille M, très bon état."
ollama ps
curl.exe http://127.0.0.1:11434/api/tags
```

Dans `ollama ps`, la colonne **PROCESSOR** doit dire **100% GPU**. Si elle affiche un mélange CPU/GPU, le modèle de texte est trop gros pour tenir avec les deux autres : en choisir un plus petit (7–8 milliards de paramètres au plus).

2.6 Contrôler la mémoire de la carte. Pendant qu'un essai tourne, `nvidia-smi` doit montrer au plus ≈ 13 Go utilisés sur 16 quand les trois modèles sont chargés.

Partie 3 · Faire tout redémarrer seul

Docker Desktop (partie 4) ne tourne que lorsqu'une session Windows est ouverte. Pour que tout reparte après un redémarrage, on ouvre la session automatiquement, puis on **verrouille l'écran aussitôt**.

3.1 Un compte Windows dédié (conseillé). Paramètres › Comptes › Autres utilisateurs › Ajouter un compte « selio » avec un mot de passe long. Fais les parties 2 à 7 en étant connecté avec ce compte.

3.2 Ouverture de session automatique. Télécharger **Autologon** de Microsoft (learn.microsoft.com, rubrique Sysinternals › Autologon). Le lancer, saisir le compte et son mot de passe, cliquer **Enable**. Le mot de passe est rangé chiffré par Windows, pas en clair.

3.3 Verrouiller l'écran juste après. Ouvrir le **Planificateur de tâches** › Créer une tâche :

- Général : nom « Verrouiller après l'ouverture automatique », compte « selio » ;
- Déclencheurs › Nouveau : « À l'ouverture de session » de « selio », **Différer la tâche de 1 minute** ;
- Actions › Nouveau : programme `rundll32.exe`, arguments `user32.dll,LockWorkStation`.

3.4 Démarrage automatique des applications. Gestionnaire des tâches › Applications de démarrage : **Ollama** doit être « Activé ». Docker Desktop sera réglé à l'étape 4.2.

3.5 Essai. Redémarrer le PC sans toucher à rien. Après 3 minutes : l'écran est verrouillé, le lama est près de l'horloge, et (après la partie 4) `docker ps` liste les conteneurs de Selio.

Partie 4 · Docker Desktop, Selio et les clés

4.1 Activer WSL 2 (le Linux intégré à Windows, utilisé par Docker). PowerShell **en administrateur** :

```
wsl --install
```

Redémarrer quand c'est demandé.

4.2 Installer Docker Desktop depuis docker.com (moteur « WSL 2 », coché par défaut). Puis dans Docker Desktop › Settings :

- General : cocher **Start Docker Desktop when you sign in to your computer** ;
- Resources : laisser les valeurs par défaut. Pour garder de la mémoire pour Ollama, créer le fichier `C:\Users\selio\.wslconfig` avec :

```
[wsl2]
memory=12GB
```

puis `wsl --shutdown` et relancer Docker Desktop.

4.3 Installer Git et Node.js. Git pour Windows (git-scm.com) et **Node.js 22 LTS** (nodejs.org). Puis, dans un nouveau PowerShell :

```
git config --global core.autocrlf input    # garde les fins de ligne Linux (indispensable pour les scripts)
corepack enable                            # active pnpm (utile à la partie 7)
```

4.4 Récupérer Selio.

```
cd C:\
git clone https://github.com/wwhabiboshdz3a7/selio.git selio
cd C:\selio
git checkout claude/celio-advertising-video-xkbv98    # la branche qui contient ces fichiers, en attendant la fusion dans main
```

Le dépôt est privé : Git ouvre une fenêtre de connexion GitHub la première fois.

4.5 Remplir le fichier des clés. C'est **le** fichier où vont toutes les clés de production :

```
cd C:\selio\deploy
Copy-Item cles-production.modele.env .env
node generer-secrets.mjs
notepad .env
```

- `generer-secrets.mjs` remplit tout seul les **A_GENERER** (mots de passe de la base, clé de chiffrement, secrets de Garage, clés des notifications…) sans rien afficher.
- Dans le Bloc-notes, remplace chaque **A_REMPLIR**. Chaque ligne est expliquée dans le fichier. Pour le tunnel (`CLOUDFLARE_TUNNEL_TOKEN`), fais d'abord la partie 5.
- `deploy\.env` est ignoré par Git : il ne partira jamais sur GitHub.

Vérifier :

```
node verifier-cles.mjs
```

Tant qu'il reste des lignes **×**, corriger et relancer. Les lignes **!** sont des choix assumés (par exemple : e-mails pas encore branchés, paiement désactivé). Ce rapport peut m'être envoyé, il ne contient aucun secret.

4.6 Copie de secours des clés. Copier `deploy\.env` dans un coffre chiffré **hors du PC** (gestionnaire de mots de passe, clé USB chiffrée). Sans `SECRETS_ENCRYPTION_KEY`, les secrets chiffrés en base sont perdus.

4.7 Vérifier que Docker voit Ollama.

```
docker run --rm curlimages/curl -s http://host.docker.internal:11434/api/tags
```

- La liste des trois modèles s'affiche : parfait, rien d'autre à faire (`OLLAMA_BASE_URL=http://host.docker.internal:11434` dans `.env`, valeur par défaut).
- Erreur « Connection refused » : Ollama n'écoute que sur `127.0.0.1` et Docker ne le voit pas. Faire écouter Ollama sur toutes les cartes réseau, **mais n'autoriser que Docker** dans le pare-feu. PowerShell en administrateur :

```
[Environment]::SetEnvironmentVariable("OLLAMA_HOST", "0.0.0.0:11434", "User")
# Supprimer les autorisations qu'Ollama aurait pu créer lui-même (fenêtre « Autoriser l'accès » de
Windows)
Get-NetFirewallApplicationFilter | Where-Object Program -like "*ollama*" | Get-NetFirewallRule |
Remove-NetFirewallRule
# N'autoriser le port 11434 que depuis Docker et WSL
New-NetFirewallRule -DisplayName "Ollama - Selio (Docker seulement)" -Direction Inbound -Protocol
TCP -LocalPort 11434 `
  -RemoteAddress 172.16.0.0/12,192.168.65.0/24 -Action Allow -Profile Any
```

Puis quitter et relancer Ollama, et refaire le test. Depuis un autre appareil du réseau (un téléphone en Wi-Fi), `http://<adresse-du-PC>:11434` ne doit **pas** répondre.

> Si un jour l'IA tourne sur un autre PC que le serveur : installer **Tailscale** sur les deux, activer « Run unattended » (icône Tailscale › Preferences), et mettre l'adresse Tailscale du PC de l'IA dans `OLLAMA_BASE_URL` (`http://100.x.y.z:11434`). Le pare-feu n'autorise alors que `100.64.0.0/10` .

Partie 5 · Nom de domaine et tunnel Cloudflare

Le tunnel est une connexion **sortante** du PC vers Cloudflare : aucun port à ouvrir sur la box, l'adresse de la maison n'est jamais exposée, et le HTTPS est fourni automatiquement.

5.1 Le domaine. Il faut un nom de domaine (ex. `selio.fr`) géré par Cloudflare : créer un compte gratuit sur cloudflare.com › **Add a domain**, puis remplacer chez le registraire (OVH, Gandi…) les serveurs DNS par les deux indiqués par Cloudflare. Compter de quelques minutes à quelques heures.

5.2 Créer le tunnel. Cloudflare › **Zero Trust** › Networks › **Tunnels** › Create a tunnel › **Cloudflared** › nom `selio-machine` › environnement **Docker**. Cloudflare affiche une commande `docker run … --token eyJ…` : **ne pas la lancer** (Docker Compose s'en charge). Copier seulement la longue valeur après `--token` dans `CLOUDFLARE_TUNNEL_TOKEN` de `deploy\.env` .

5.3 Les deux adresses publiques. Dans le tunnel › **Public hostnames** (ou « Routes ») › Add :

SOUS-DOMAINE	TYPE	ADRESSE DU SERVICE	RÉGLAGE EN PLUS
`api.selio.fr`	HTTP	`api:8787`	aucun
`photos.selio.fr`	HTTP	`garage:3900`	Additional application settings › HTTP Settings › **HTTP Host Header** = `photos.selio.fr`

L'en-tête « Host » est indispensable pour les photos : chaque adresse de photo est signée avec ce nom.

5.4 Pas de cache pour les photos. Cloudflare › ton domaine › Caching › **Cache Rules** › Create rule : nom « Photos sans cache », « Hostname equals `photos.selio.fr` », action **Bypass cache**. Les photos des clients sont privées : aucune copie ne doit rester chez Cloudflare.

5.5 Dans `deploy\.env` : `API_PUBLIC_URL=https://api.selio.fr` , `STORAGE_ENDPOINT=https://photos.selio.fr` , `PUBLIC_WEB_URL` et `CORS_ORIGINS` = l'adresse du site. Relancer `node verifier-cles.mjs` .

5.6 Dans Netlify (Site configuration › Environment variables) : `VITE_API_BASE_URL=https://api.selio.fr` , puis redéployer le site.

À savoir : les échanges passent par les serveurs de Cloudflare (entreprise américaine, serveurs européens pour les visiteurs français), mais **les données restent stockées sur ce PC**, en France. Si on préfère s'en passer : ouvrir les ports 80 et 443 de la box vers le PC et utiliser Caddy (`deploy/Caddyfile`). C'est plus exposé et ça demande une adresse IP fixe.

Partie 6 · Démarrer et vérifier

6.1 Démarrer. Dans PowerShell :

```
cd C:\selio\deploy
docker compose -f docker-compose.prod.yml -f docker-compose.machine.yml up -d --build
```

La première fois, la construction prend 5 à 10 minutes. Ensuite :

```
docker compose -f docker-compose.prod.yml -f docker-compose.machine.yml ps -a
```

Attendu : `postgres`, `redis`, `garage`, `api`, `worker`, `tunnel` en **Up** ; `migrate` et `garage-init` en **Exited (0)** (ils font leur travail puis s'arrêtent).

6.2 Lire les journaux de démarrage.

```
docker compose -f docker-compose.prod.yml -f docker-compose.machine.yml logs garage-init migrate
docker compose -f docker-compose.prod.yml -f docker-compose.machine.yml logs api | Select-String
"msg"
```

- `garage-init` : se termine par « pré-vol CORS vérifié · Garage prêt » ;
- `migrate` : « 12 migration(s) appliquée(s) » la première fois ;
- `api` : « Notifications Web Push activées », **pas** de « Ollama injoignable ». Les avertissements « EMAIL_PROVIDER=noop » et « Paiement non configuré » sont normaux tant que l'e-mail et Revolut ne sont pas branchés.

6.3 Depuis l'extérieur. Sur le téléphone, **en 4G** (pas en Wi-Fi), ouvrir `https://api.selio.fr/api/health` : la page affiche `{"ok":true,…}`.

6.4 Santé de l'IA. Sur le PC (le jeton est dans `deploy\.env`, ligne `AI_HEALTH_TOKEN`) :

```
$t = (Select-String -Path C:\selio\deploy\.env -Pattern '^AI_HEALTH_TOKEN=
(.+)$').Matches.Groups[1].Value
curl.exe -s -H "x-health-token: $t" https://api.selio.fr/api/ai/health
```

Attendu : `"status":"healthy"` et les trois modèles `"present":true`.

6.5 Dans l'application. Créer ton compte sur le site avec l'adresse mise dans `OPERATOR_EMAILS` (elle donne l'accès à l'administration), puis Paramètres › Assistant IA : **Opérationnel**. Paramètres › Notifications : activer sur le téléphone, puis « notification de test ».

Où en est le code : le stockage des photos est prêt et testé (envoi et lecture signés, refus de tout le reste), mais l'application web n'envoie pas encore ses photos vers Garage : elle les garde dans la fiche de l'article. Ce branchement côté navigateur est décrit dans `docs/STOCKAGE.md` et reste à faire avant la bêta.

Partie 7 · Mesurer l'IA

Ces mesures remplacent les estimations (« ≈ 15 réponses par minute », « ≈ 3,6 s par photo ») par les vrais chiffres de cette machine. Elles décident des quotas des offres.

```
cd C:\selio
pnpm install --frozen-lockfile
$env:AI_PROVIDER = "ollama"; $env:OLLAMA_BASE_URL = "http://127.0.0.1:11434"; $env:OLLAMA_MODEL =
"qwen2.5:7b-instruct"

pnpm ai:diagnose                                  # code 0 attendu : les trois modèles présents

$env:BENCH_SECONDS = "120"; $env:BENCH_WATTS = "300"
pnpm ai:bench                                     # réponses aux acheteurs par minute, p50 / p95

$env:BENCH_MODE = "photo"; $env:BENCH_PHOTO = "C:\selio\photo-test.jpg"; $env:BENCH_PHOTOS = "4"
pnpm ai:bench                                     # secondes par photo Studio

$env:BENCH_MODE = "embed"; $env:BENCH_BATCH = "16"
pnpm ai:bench                                     # textes par seconde (modèle des prix)
```

- `photo-test.jpg` : n'importe quelle photo d'article prise au téléphone, en JPEG, réduite à environ 1 000 pixels de large.
- `BENCH_WATTS` : la consommation du PC en charge, lue sur un wattmètre. À défaut, lancer `nvidia-smi --query-gpu=power.draw --format=csv -l 1` pendant la mesure, qui donne celle de la carte seule.
- Reporter les chiffres dans le tableau « Mesures sur la machine » de `docs/OLLAMA-SETUP.md`, et me les envoyer : ils ne contiennent rien de secret.

Partie 8 · Au quotidien

ACTION	COMMANDE (DANS C:\SELIO\DEPLOY)
Mettre à jour Selio	`git pull`, puis la commande de démarrage 6.1 (les migrations passent toutes seules)
Voir ce qui tourne	`docker compose -f docker-compose.prod.yml -f docker-compose.machine.yml ps -a`
Suivre l'API en direct	`docker compose -f docker-compose.prod.yml -f docker-compose.machine.yml logs -f api`
Arrêter proprement	`docker compose -f docker-compose.prod.yml -f docker-compose.machine.yml down`
Mettre à jour un modèle	`ollama pull <nom>`

> **Jamais `down -v`** : l'option `-v` supprime les volumes, c'est-à-dire **la base des clients et toutes les photos**.

Sauvegarde. Avant d'accueillir de vrais clients, mettre en place la sauvegarde chiffrée de la nuit (`docs/SAUVEGARDE.md`). Sous Windows, les scripts `scripts/sauvegarde/*.sh` se lancent dans **Ubuntu (WSL)**, installé à l'étape 4.1, avec `BACKUP_PG_DOCKER_CONTAINER=deploy-postgres-1`. Pour les photos, créer une clé Garage en lecture seule : `docker compose -f docker-compose.prod.yml -f docker-compose.machine.yml exec garage /garage key create selio-sauvegarde`, puis `… exec garage /garage bucket allow --read selio-photos --key selio-sauvegarde`.

En cas de problème

CE QU'ON VOIT	CAUSE PROBABLE	QUE FAIRE
`verifier-cles.mjs` affiche des ×	une valeur manque ou a un mauvais format	lire le message à droite de la ligne, corriger `.env`, relancer
« Ollama injoignable » dans les journaux de l'API	Ollama arrêté, ou non visible depuis Docker	lama présent près de l'horloge ? Sinon le relancer. Puis refaire le test 4.7
`ollama ps` affiche du CPU	le modèle de texte est trop gros avec les deux autres	choisir un modèle de 7–8 milliards de paramètres au plus
`docker compose` dit « défini dans .env »	une variable obligatoire est vide	`node verifier-cles.mjs` indique laquelle
`garage-init` en « Exited (1) »	clé de stockage au mauvais format, ou changée après le premier démarrage	lire `logs garage-init` : le message dit quoi faire
`https://api.selio.fr` ne répond pas	tunnel arrêté ou mal configuré	`logs tunnel` ; dans Cloudflare, le tunnel doit être « Healthy » et l'adresse du service `api:8787`
Photos en erreur 403 « signature »	en-tête Host manquant sur `photos.selio.fr`	régler « HTTP Host Header » (étape 5.3)
Rien ne tourne après un redémarrage	session non ouverte, ou Docker Desktop non lancé	refaire la partie 3, et cocher le démarrage de Docker Desktop (4.2)

Ce qui reste fermé, volontairement

Selio Cloud, Radar Auto et l'achat réel restent **fermés** (`CLOUD_ENABLED=false`, `RADAR_AUTO_ENABLED=false`, `RADAR_REAL_PURCHASE=false`) tant que l'avocat n'a pas rendu son avis écrit. Le paiement reste en **bac à sable** tant que le parcours complet n'est pas validé (`docs/CLES-ET-CONFIGURATION.md` §5). Ne pas les ouvrir pour « essayer » sur la machine de production.